Hierarchical LLM-Based Planning and Replanning for Autonomous Multi-Drone Systems

Ákos Varga, 202403575

Master's Thesis, Computer Engineering

Spring 2026

Advisor: Andriy Sarabakha

AARHUS UNIVERSITY

Abstract

This thesis investigates the use of Large Language Models (LLMs) for high-level planning and adaptive replanning in heterogeneous multi-drone systems. The system uses a hierarchical architecture in which a cloud-based planning pipeline generates mission plans from natural-language instructions, while a smaller locally deployable model performs task admission.

The planning pipeline decomposes a natural-language mission description into atomic subtasks, assigns them to drones according to their capabilities, and generates an execution schedule. Before and during execution, the task admission module determines whether assigned subtasks are feasible given the drone's state. If a drone failure or task rejection occurs, the affected subtasks are returned to the cloud-based planner for rescheduling.

The system was evaluated in three parts. First, several cloud-based LLMs were compared for task decomposition, allocation, and scheduling using missions with one to ten subtasks. The generated schedules were evaluated against a vehicle routing problem baseline in terms of schedule quality, validity, and inference time. GPT-5-mini was selected for the cloud-based planner because it provided the best trade-off between quality, latency, and cost. Second, seven quantized locally deployable models were evaluated for task admission using scenarios involving drone-state constraints, link-quality issues, and flight-time limitations. The `qwen3:1.7b` model achieved the highest reliability and was selected for the local task admission module.

Finally, the selected components were integrated into a simulation environment containing heterogeneous drones and diverse target objects, while also simulating battery drainage, link-quality variation, and drone failures. The simulation evaluation demonstrated that the system can generate mission schedules, perform task admission checks, detect runtime failures, reschedule affected subtasks, and continue execution with the remaining drones. The physical Parrot Anafi drone platform and laboratory integration setup are also described to map the symbolic planner skills to real drone capabilities.

The results show that LLMs can support flexible high-level mission planning and replanning when combined with structured task representations, capability-based allocation, and execution-time feasibility checks. However, the complete closed-loop architecture was evaluated in simulation rather than deployed on physical drones, leaving full real-world validation as future work.

Contents

List of Figures

List of Tables

Abbreviations

ALM	Audio Language Model
API	Application Programming Interface
CoT	Chain-of-Thought
CPU	Central Processing Unit
DSP	Digital Signal Processing
FHD	Full High Definition
GPU	Graphics Processing Unit
GPS	Global Positioning System
HD	High Definition
IMU	Inertial Measurement Unit
LiDAR	Light Detection and Ranging
LLM	Large Language Model
MAS	Multi-Agent System
MILP	Mixed-Integer Linear Programming
mTSP	Multiple Traveling Salesman Problem
NPU	Neural Processing Unit
ROS	Robot Operating System
SAR	Search and Rescue
UAV	Unmanned Aerial Vehicle
VLM	Vision-Language Model
VRP	Vehicle Routing Problem
VTOL	Vertical Takeoff and Landing

Chapter 1

Introduction

1.1 Problem Statement

Autonomous drone systems are increasingly deployed in applications such as infrastructure inspection [1], surveillance [2], and environmental monitoring [3]. While reliable low-level flight control and sensing capabilities have been widely developed, expressing high-level mission objectives in a way that is both intuitive for human operators and executable by autonomous systems remains a challenge.

Existing approaches rely on predefined workflows or formal planning models, which require expert knowledge and lack flexibility when faced with dynamic, changing environments. As a result, translating human instructions into concrete mission plans remains a bottleneck.

Recent advances in Large Language Models (LLMs) offer a promising approach for bridging this gap. LLMs can interpret natural-language instructions and infer task structure, enabling them to transform human mission descriptions into structured plans that autonomous systems can execute. This capability makes them well suited for high-level reasoning tasks such as task decomposition, allocation, and scheduling in multi-agent systems. This thesis proposes the use of LLMs to generate executable plans for multi-drone missions. Given a natural-language mission description, the system decomposes the task into atomic subtasks, allocates them to drones according to their capabilities, and generates a structured execution schedule.

To address the challenges of real-world deployment, the system adopts a hierarchical architecture in which cloud-based LLMs perform high-level mission planning, while smaller locally deployed models perform task admission checks and trigger replanning when assigned tasks become infeasible. The goal is to develop a flexible and robust framework capable of translating human intent into coordinated multi-drone behavior. In this architecture, the LLM is used only for high-level mission planning and task admission. It does not directly control safety-critical low-level flight behavior, which remains handled by the execution layers.

Compared with existing LLM-based robotic planning systems, the novelty of this thesis is the integration of natural-language multi-drone mission planning with explicit schedule generation, capability-aware task allocation, local task admission, and replanning of unfinished subtasks. While prior systems have explored LLM-based task decomposition, robot coordination, or reactive replanning, they often do not produce inspectable execution schedules with explicit timing information, or they do not combine scheduling with execution-time feasibility checks. This thesis addresses this gap by generating per-drone schedules from natural-language mission descriptions, checking assigned subtasks locally before and during execution, and returning unfinished or infeasible subtasks to the planner for rescheduling.

The implementation is available to support reproducibility and further development. [1]

1.2 Research Goals

The primary goal of this thesis is to investigate the feasibility and effectiveness of using Large Language Models for high-level planning and adaptive replanning in multi-drone systems. While LLMs have demonstrated strong capabilities in natural-language understanding and reasoning, their applicability to real-time, safety-critical autonomous systems remains an open question.

This thesis addresses the following research goals:

- To evaluate how effectively LLMs can translate natural-language mission descriptions into structured, executable task plans for multi-drone systems.
- To investigate the ability of LLMs to perform task decomposition, allocation, and scheduling in environments with heterogeneous agents and constrained resources.
- To assess the robustness of LLM-based planning when operating in dynamic environments that require replanning due to failures, delays, or environmental changes.
- To analyze the trade-offs between cloud-based and locally deployed models in terms of latency, reliability, and responsiveness during mission execution.
- To identify the limitations of LLM-based planning approaches and determine under which conditions they outperform or fall short of traditional planning methods.

1.3 Thesis Structure

This thesis is organized as follows:

Chapter 2 provides the technical background required for the thesis. It introduces autonomous drone systems, multi-agent coordination and planning, vehicle routing concepts, Large Language Models, and related work on LLM-based robotic and multi-agent planning.

[1] The source code developed for this thesis is available at: `https://github.com/Akos-Varga/llm-drone-planning-replanning`.

Chapter 3 presents the proposed methodology and system architecture. It describes the LLM-based planning pipeline and also explains the local task admission module, the replanning mechanism, and the software architecture used to coordinate the planner and drone workers.

Chapter 4 presents the evaluation results, including cloud-based planner comparisons against a vehicle routing problem baseline, local task admission evaluation, an end-to-end simulation evaluation across multiple mission scenarios, and a representative simulation case study. It also describes the physical Parrot Anafi platform and laboratory setup for future real-world deployment.

Chapter 5 concludes the thesis by summarizing the main findings, discussing the limitations of the proposed approach, and outlining directions for future work.

Chapter 2

Background

2.1 Autonomous Drone Systems

Autonomous drone systems are becoming increasingly prominent in applications that are otherwise time-consuming or dangerous when performed using traditional methods. Such applications include infrastructure inspection, for example monitoring bridge structures [4], environmental monitoring such as wildlife surveys [3], and search and rescue (SAR) missions, where drones can assist in locating people after disasters [5]. Unmanned Aerial Vehicles (UAVs) are also widely used in surveillance and security applications, including border monitoring and crowd observation [6]. In addition, drones have been successfully deployed for logistics and delivery tasks, such as Amazon Prime Air [7], as well as real-world medical transport solutions, including long-range delivery of blood samples in Denmark [8]. Large-scale industrial and energy infrastructures can likewise be efficiently monitored using UAVs [9].

These diverse applications highlight the potential of UAVs; however, understanding the characteristics and capabilities of different drone platforms is essential.

Multirotor drones are widely used due to their ability to hover, perform vertical takeoff and landing (VTOL), and navigate precisely in constrained environments. These capabilities make them particularly suitable for urban operations such as inspection and delivery. In contrast, fixed-wing drones are designed for long-endurance missions and can cover large areas efficiently, making them well-suited for large-scale mapping and environmental monitoring tasks. However, they lack hovering capability and require more space for takeoff and landing, limiting their use in high-precision or urban scenarios.

The suitability of a drone for a given mission depends heavily on the payload it carries. RGB cameras provide visual data for inspection, surveillance, and mapping tasks, and can also be used for motion estimation through visual odometry [10]. Thermal cameras mounted on UAVs have been successfully applied in infrastructure inspection, for example to detect subsurface delamination in bridge decks using infrared thermography [4]. LiDAR-equipped UAVs enable accurate three-dimensional mapping of environments, for instance in forestry applications where canopy structure can be estimated from aerial point clouds [11]. In addition, onboard sensors such as Inertial Measurement Units (IMUs), GPS, and barometers are used for state estimation, including position, velocity, and altitude.

In multi-drone systems, payload diversity introduces functional heterogeneity, meaning that not all drones are capable of executing all tasks. This has direct implications for task allocation, as tasks must be matched with drones that possess the required sensing and operational capabilities.

Many modern UAV applications rely on artificial intelligence (AI) for tasks such as perception, navigation, and decision-making. However, developers often face strict hardware constraints in drone systems. While cloud-based AI can provide significant computational resources, it introduces latency and dependence on network availability. The emergence of edge AI enables real-time decision-making by allowing drones to process sensor data locally and operate independently of external infrastructure [12]. These platforms often support hardware acceleration through GPUs, DSPs, or NPUs, enabling efficient execution of machine learning models [13]. This is particularly relevant for deploying compact and quantized large language models (LLMs) directly onboard drones.

2.2 Multi-Agent Systems

Multi-agent systems (MAS) consist of multiple autonomous agents that interact in a shared environment to achieve a collective goal [14]. In the context of autonomous drone systems, each drone can be modelled as an autonomous agent with its own sensing and decision-making capabilities. Deploying heterogeneous UAV agents increases the number of available data modalities, expands the area that the drone swarm can inspect, and provides backup options in case of drone failures, thereby increasing the mission success rate in contrast to a single-agent solution [15]. The key challenges in multi-agent systems are the efficient coordination between agents and the creation of a mission plan for heterogeneous agents, particularly under constraints such as limited communication, energy, and computation.

2.2.1 Coordination in Multi-Agent Systems

Two common architectural approaches for coordination in multi-agent systems are centralized and decentralized coordination. In a centralized architecture, a base station or ground control station is responsible for critical mission decisions and coordinates the behavior of the agents. Since the central unit is often equipped with greater computational resources than the individual agents, centralized approaches can support more complex planning and optimization. However, this architecture introduces a single point of failure: if the base station fails or communication with it is lost, the mission may be compromised.

In a decentralized architecture, each agent is capable of local decision-making and may communicate directly with other agents. This can improve robustness, since the system can remain operational even if individual agents fail. Decentralized systems may also scale better as new agents are added, since control is not concentrated in a single central unit. However, coordination becomes more challenging because agents must reach decisions using local information and distributed communication mechanisms.

Several mechanisms can support decentralized coordination. Leader election methods dynamically select a coordinating agent responsible for organizing group decisions. Auction-based methods allocate tasks by allowing agents to bid based on cost, capability, or availability, with each task assigned to the agent that submits the best bid according to the chosen objective. Voting mechanisms allow agents to collectively choose between alternatives, while bargaining and negotiation approaches enable agents to iteratively exchange proposals until an agreement is reached.

Hybrid approaches combine elements of both centralized and decentralized coordination. For example, a ground station may perform high-level mission planning, while individual agents retain autonomy to make local decisions based on their current state and surrounding environment. Such architectures can exploit the computational resources of centralized systems while preserving some of the robustness and adaptability of decentralized systems.

2.2.2 Planning in Multi-Agent Systems

In multi-agent systems with heterogeneous agents, the task allocation problem consists of assigning tasks to agents under given constraints, such as energy, capability, and communication limitations, while optimizing a global objective, for example minimizing mission time or maximizing coverage. Several approaches to multi-agent planning have been identified in the literature [16].

One approach is centralized planning for distributed execution, where a central planner constructs a global plan for all agents. This plan specifies both the allocation of tasks and the ordering of actions, and is subsequently distributed to individual agents for execution. While this approach simplifies coordination by leveraging a global view of the system, it relies on a central entity and is therefore susceptible to single points of failure.

An alternative approach is distributed planning with centralized plan generation, where multiple agents collaborate to construct a single global plan. In this case, agents may contribute specialized knowledge or capabilities during the planning phase, but the resulting plan is still shared among agents for execution. This approach can improve flexibility in plan generation, although it still assumes the existence of a coherent global plan.

The most decentralized approach is distributed planning for distributed execution, where each agent independently generates its own plan while coordinating with others during execution. In such systems, agents may only possess partial knowledge of the overall mission and must dynamically resolve conflicts and dependencies. Coordination in this setting often relies on negotiation, communication, or local decision-making mechanisms. As a result, a fully global plan may never exist, and system behavior emerges from the interaction of individual agent plans.

In general, centralized planning approaches are simpler to design and can produce globally consistent plans due to their access to complete system information. However, decentralized approaches offer greater robustness and scalability, as they avoid reliance on a single coordinating entity. The choice between these approaches depends on the requirements of the system, including the need for global optimality, robustness to failures, and adaptability to dynamic environments.

Many multi-agent planning problems can be formulated as optimization problems. A common abstraction is the multiple traveling salesman problem (mTSP), where multiple agents must visit a set of locations while minimizing a cost function. In this formulation, tasks correspond to nodes, agents correspond to salesmen, and decision variables determine the assignment and ordering of tasks for each agent. While mTSP provides a useful abstraction, more realistic formulations are given by the Vehicle Routing Problem (VRP). In the VRP, the objective is to determine optimal routes for multiple vehicles that must visit a set of locations. In the special case of a single vehicle, the problem reduces to the classical Traveling Salesperson Problem (TSP). The notion of optimality in VRP depends on the application context. A common objective is to minimize the total distance traveled by all vehicles; however, this may lead to unbalanced solutions where a single agent performs most tasks. An alternative objective is to minimize the length of the longest route, which promotes balanced workload distribution and reduces overall mission completion time. Furthermore, VRP formulations can incorporate additional constraints, such as capacity limits and time windows, which allow modeling of practical considerations like energy constraints and task deadlines in multi-drone systems.

Different solution approaches exist for such problems. Exact methods, such as Mixed-Integer Linear Programming (MILP), can provide optimal solutions but often suffer from high computational complexity and limited scalability. In contrast, heuristic and metaheuristic methods, such as genetic algorithms, can provide faster solutions and are more suitable for large-scale problems, albeit without guarantees of optimality [17].

Finally, real-world multi-agent systems must handle dynamic changes, such as agent failures. In such cases, replanning is required, which can be performed either centrally by a global planner or locally by individual agents. Hybrid approaches may combine both strategies, allowing systems to balance global optimality with robustness and adaptability.

2.3 Large Language Models

2.3.1 Fundamentals of Large Language Models

Large Language Models (LLMs) are computational systems designed to generate and understand natural language by modeling the statistical structure of text. At their core, LLMs estimate the probability distribution of the next token given a sequence of preceding tokens. Formally, given a context of $\omega_1, \omega_2, \ldots, \omega_{t-1}$, the model predicts the next token w_t, thereby enabling both language understanding and generation. The key idea that enables modern LLMs is pretraining, where models learn linguistic structure and world knowledge by repeatedly predicting the next token over very large text corpora.

2.3.2 Transformer Architecture

Modern LLMs are implemented using the Transformer architecture introduced in Attention is All You Need [18]. The core component of the Transformer is the self-attention mechanism, which enables the model to weigh the importance of different tokens relative to each other when generating representations. Given an input sequence, attention computes interactions between all token pairs, allowing the model to capture long-range dependencies efficiently.

2.3.3 Model Scaling and Deployment Considerations

Large Language Models vary significantly in size, ranging from millions to hundreds of billions of parameters. While larger models generally achieve higher performance due to increased representational capacity, they also require substantial computational resources, including high memory bandwidth and specialized hardware such as GPUs. This poses a challenge for deployment in resource-constrained environments, such as embedded systems, mobile devices, or robotic platforms.

A common approach to address these limitations is quantization, where model weights are represented with reduced numerical precision (e.g., 8-bit or 4-bit instead of 32-bit floating point). Quantized LLMs significantly reduce memory usage and computational requirements, enabling deployment on edge devices while maintaining acceptable performance.

2.3.4 LLMs in Multi-Agent Systems

Large Language Models (LLMs) have been successfully applied in various domains, including multi-agent systems, due to their strong reasoning abilities, contextual awareness, and generalization capabilities. In this context, their applications can be broadly categorized into communication, perception, planning, and control, following [19].

LLMs are particularly effective at handling the ambiguity of natural language and can be used for both *interpretation* and *grounding*. Interpretation refers to converting natural-language into structured representations, such as code or formal planning languages (e.g., PDDL). Grounding involves mapping human instructions to objects, actions, or processes that can be recognized and executed by agents.

Perception enables agents to operate in real-world environments. Language models have been extended to multimodal settings by integrating visual and auditory inputs. These models, such as Vision-Language Models (VLMs) and Audio Language Models (ALMs), process non-textual data alongside language, allowing agents to detect objects and interpret signals from their surroundings.

In planning, traditional approaches rely on predefined algorithms that must be adapted whenever the environment or objectives change. In contrast, LLM-based planning through prompting allows for flexible task specification and the incorporation of environmental feedback. However, due to the risk of hallucinations, LLM-generated plans should be validated against system constraints or combined with external planning methods.

For control, LLMs can be used in two main ways. In the direct approach, the model maps natural-language instructions directly to control commands or actions. In the indirect approach, the model generates higher-level outputs such as subgoals or plans, which are then executed by separate control systems. The indirect approach is generally more flexible and better suited for complex and multi-agent scenarios, although it still requires reliable grounding of language into physical actions.

2.4 Related Work

Recent work has increasingly explored the use of Large Language Models (LLMs) for robotic planning, multi-agent coordination, and autonomous task execution. LLMs are particularly attractive in this context because they can interpret high-level natural-language instructions and transform them into structured plans, executable code, or robot-specific actions. However, robotic systems operate under physical constraints, including limited energy, sensing capabilities, communication quality, and environmental uncertainty. For this reason, LLM-based planning systems must not only generate reasonable plans, but also ensure that these plans are feasible, executable, and adaptable during execution.

Among early LLM-based planners for autonomous agents, SayCan [20] introduces a framework that grounds language model reasoning in real-world robot capabilities. In this approach, an LLM generates candidate low-level skills from a high-level natural-language instruction. Each skill is evaluated using two complementary scores: (i) a language model score, which measures how relevant the skill is for completing the task, and (ii) an affordance score, which estimates the feasibility of executing the skill given the robot's current state and environment. These scores are combined to select the most appropriate action, and the process is iteratively repeated until the task is completed. This work demonstrates that LLMs can be effectively applied to robotics when their outputs are constrained by physical feasibility. However, SayCan focuses primarily on single-agent sequential decision-making and does not address task allocation, scheduling, or coordination between multiple robots. Furthermore, while feasibility is considered at the skill-selection level, the framework does not generate explicit multi-agent schedules or perform mission-level replanning when execution conditions change.

In the context of UAVs, Wang et al. [21] introduced the GSCE framework, which combines guidelines, skill APIs, constraints, and examples within a single prompt. By leveraging step-by-step Chain-of-Thought (CoT) reasoning together with few-shot examples, the system generates executable code for a single drone. The framework is relevant because it shows how LLMs can map natural-language instructions to executable drone actions while respecting predefined APIs and operational constraints. This is important for UAV applications, where generated plans must be compatible with the available control interface. However, GSCE is mainly designed for single-drone operation. It does not address the additional challenges that arise in multi-drone missions, such as assigning subtasks to different drones, coordinating parallel execution, balancing workload, or replanning when one drone becomes unable to complete its assigned task. Thus, while SayCan and GSCE highlight the potential of LLMs for grounded reasoning and executable plan generation, they do not fully address multi-agent scheduling or execution-time adaptation.

Extending these ideas to multi-agent systems, RoCo [22] introduces a collaborative reasoning framework for multi-robot coordination. In RoCo, each robot is associated with an LLM-based agent that has access to its own capabilities, local observations, and partial knowledge of the environment. These agents communicate through a structured dialogue process, iteratively proposing, refining, and critiquing task plans. Through this interaction, the system converges to a shared global plan, which is then decomposed and distributed among the robots for execution. Additionally, RoCo leverages LLMs during execution by generating 3D waypoints, enabling more flexible behavior. RoCo is important because it demonstrates how LLMs can support communication and collaborative reasoning between multiple robots rather than relying on a single planner. However, the generated plans are primarily the result of dialogue-based coordination and are not represented as explicit schedules. As a result, it is difficult to evaluate schedule quality using objective metrics such as mission completion time, workload balance, or synchronization constraints. Furthermore, although the system supports flexible execution, it does not focus on structured replanning based on explicit feasibility checks of assigned subtasks.

Building on multi-agent coordination, SMART-LLM [23] introduces a structured framework for task planning in multi-agent systems, generating executable mission plans through a three-stage pipeline, each implemented by a dedicated LLM agent. In the first stage, *task decomposition*, the LLM converts a high-level natural-language instruction into a set of independent subtasks. Each subtask is represented as a sequence of actions, while incorporating contextual information about the environment. The second stage, *coalition formation*, considers the available robots and their capabilities, and assigns groups of robots to each subtask such that every coalition collectively satisfies the required skill set for execution. In the final stage, *task allocation*, the system generates executable code for each robot based on the assigned subtasks and coalitions, resulting in a distributed execution plan. This work highlights the importance of decomposing complex planning problems into modular steps, where each stage focuses on a specific aspect of the planning process, and demonstrates the effectiveness of few-shot prompting in improving the quality and consistency of LLM outputs. SMART-LLM is closely related to this thesis because it also separates high-level mission planning into decomposition and allocation stages. However, the framework does not explicitly focus on temporal scheduling, schedule optimization, or runtime replanning. The resulting plans specify which robots should perform which tasks, but they do not provide a detailed execution schedule with explicit timing information. Moreover, the system does not include a dedicated task admission mechanism for checking whether assigned subtasks remain feasible before and during execution. This limits its ability to detect infeasible assignments and revise the plan when drone states or communication conditions change.

To enable adaptive behavior in dynamic environments, CoMuRoS [24] is another framework that incorporates a hierarchical architecture with event-driven replanning. The system employs a central LLM, referred to as the Task Manager, for high-level reasoning, including interpreting natural-language goals, decomposing tasks, allocating subtasks, and updating plans based on environmental feedback. Each robot is equipped with a local LLM that translates assigned tasks into executable Python code and performs event classification, distinguishing between relevant and irrelevant observations. Relevant events are communicated back to the Task Manager to trigger replanning. This makes CoMuRoS particularly relevant to this thesis because it explicitly considers replanning as part of the execution process. The framework demonstrates that LLM-based systems can adapt to changes in the environment and revise plans in response to runtime events.

However, CoMuRoS primarily focuses on feasibility and reactive coordination rather than explicit schedule generation. Although the system demonstrates strong adaptability and has been validated in real-world scenarios with heterogeneous robots, including drones and quadrupeds, the generated plans lack explicit temporal reasoning. Furthermore, the framework does not incorporate a clear objective function, such as minimizing total mission time, leading to potentially suboptimal utilization of resources. The replanning process is therefore mainly event-driven and reactive, rather than being connected to an explicit schedule that can be evaluated, updated, and compared against an optimization baseline. This leaves open the question of how LLM-based replanning can be integrated with explicit scheduling in multi-drone missions.

Complementary to replanning-focused approaches, AutoHMA-LLM [25] proposes a hierarchical architecture that combines cloud-based LLMs, device-level LLMs, and classical control algorithms to address task coordination and scheduling. In this framework, the cloud LLM performs global task decomposition, allocation, and scheduling, while device-level LLMs translate tasks into robot-specific instructions executed by classical control methods. The

Table 2.1: Comparison of LLM-based multi-agent systems.

	SayCan	GSCE	RoCo	SMART-LLM	CoMuRoS	AutoHMA	Ours
Year	2022	2025	2023	2024	2025	2025	2026
Multi-Robot	✗	✗	✓	✓	✓	✓	✓
Heterogeneous	✓	✗	✓	✓	✓	✓	✓
Framework	Centralized	Centralized	Decentralized	Centralized	Hybrid	Hybrid	Centralized
Replanning	✗	✗	✓	✗	✓	✓	✓
Hardware Integration	✓	✗	✓	✗	✓	✗	✓†
Scheduling	✗	✗	✗	✗	✗	✓	✓
Objective Function	✗	✗	✗	✗	✗	✓	✓

† Physical integration setup only; full closed-loop evaluation was performed in simulation.

system explores multiple coordination strategies and demonstrates that a centralized hybrid model achieves the best trade-off between efficiency and performance through continuous feedback between system layers. AutoHMA-LLM is relevant because it explicitly considers scheduling and hierarchical coordination, which are important for multi-agent systems where high-level reasoning and low-level execution must be combined. However, although the framework formulates scheduling as an optimization problem, it does not produce explicit schedules that can be directly inspected or executed. Additionally, the system is evaluated primarily in simulation environments, without validation on real robotic platforms, leaving its performance under real-world conditions unverified. The framework also does not focus on task admission as a mechanism for detecting infeasible subtasks before or during execution.

As shown in Table 2.1, existing research demonstrates the growing capabilities of LLM-based multi-agent systems in task decomposition, coordination, scheduling, and adaptive replanning. However, these capabilities are often addressed separately, and existing systems typically do not combine explicit schedule generation with execution-time feasibility checking and replanning in a single multi-drone architecture.

This creates a research gap for LLM-based multi-drone planning systems that combine natural-language task decomposition, capability-aware allocation, explicit scheduling, and execution-time replanning. In particular, there is a need for systems that can not only generate an initial mission plan, but also evaluate whether assigned subtasks remain feasible during execution and revise the schedule when drone states, communication conditions, or task requirements change. This thesis addresses this gap by developing a planning pipeline that produces explicit execution schedules and integrates a task admission module for detecting infeasible subtasks before and during execution, enabling replanning in response to runtime changes.

Chapter 3

Methodology

3.1 System Overview

The core of the system is a cloud-based multi-drone task scheduling pipeline. Given a high-level mission command from a user, the system creates an explicit schedule for a team of heterogeneous drones. The pipeline consists of three stages: task decomposition, task allocation, and task scheduling. First, the task decomposition stage converts the natural-language mission into a list of atomic subtasks. Second, the task allocation stage assigns each subtask a set of suitable candidate drones based on the required skill for the subtask and the capabilities of each drone. Finally, the scheduling stage selects the final drone for each subtask and determines the temporal order of execution, taking into account travel time and task execution duration, while aiming to minimize the overall mission makespan.

The output of the pipeline is a per-drone schedule with departure, arrival, and finish times for each subtask. This representation enables coordinated multi-drone operation and allows the generated plan to be inspected before flight. An overview of the pipeline with an example task is shown in Figure 3.1. For readability, the prompt boxes show simplified versions of the full prompts used in the experiments.

The system also supports replanning in case of drone or task failures, such as critically low battery level, poor connection, or a crash. In such cases, the planner updates the set of available drones and regenerates the schedule for unfinished or interrupted subtasks.

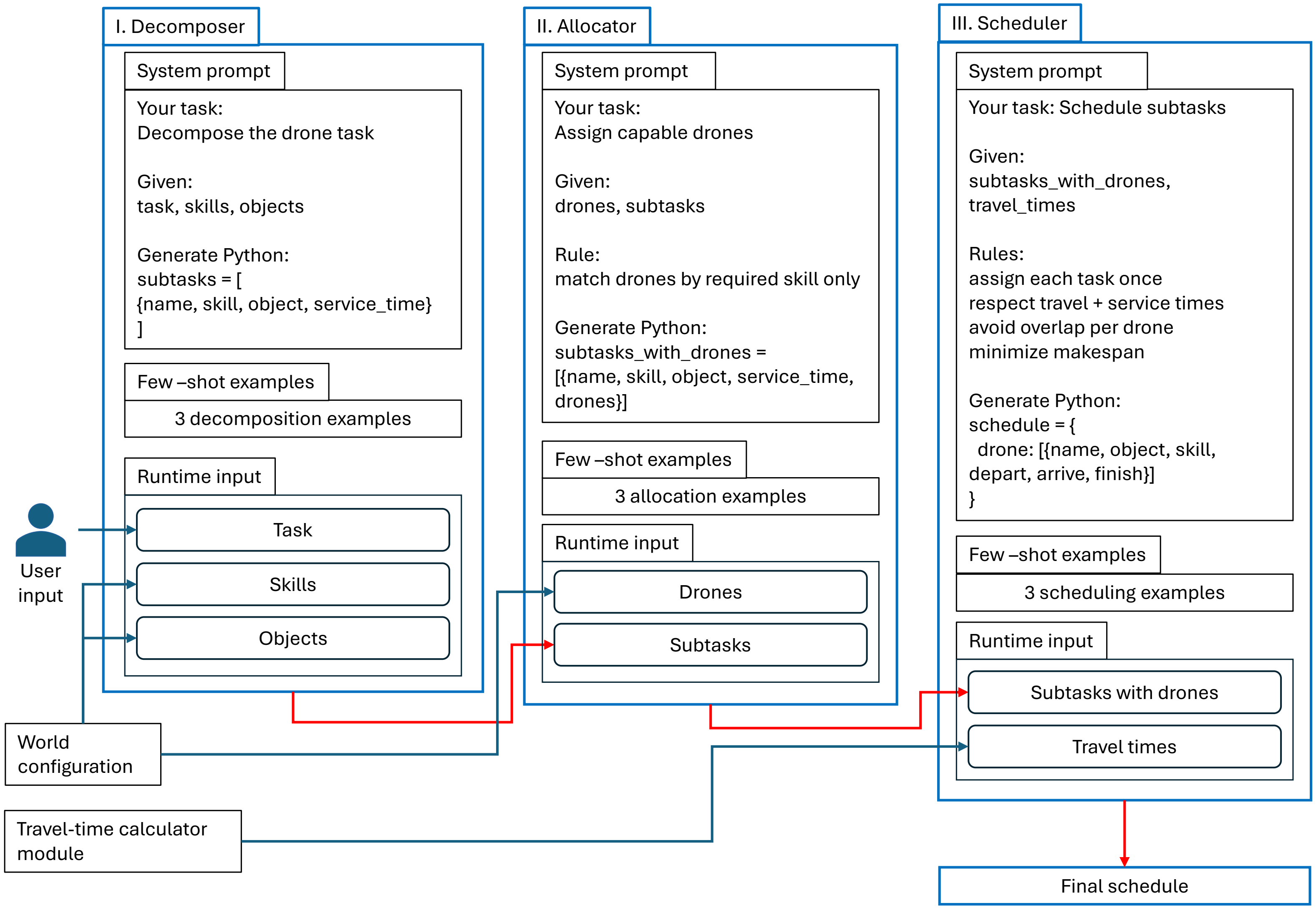


Figure 3.1: Proposed LLM-based multi-drone planning pipeline. Blue arrows represent input/context data, and red arrows represent LLM-generated outputs.

3.2 Task Decomposition

The task decomposition phase transforms high-level mission commands into a structured set of subtasks. A mission instruction may contain several objectives, such as inspecting different structures, capturing images, recording videos, or collecting environmental measurements. The atomic subtasks generated by the decomposer can later be assigned to drones and scheduled independently.

The decomposition stage is performed using an LLM prompted with the available environmental objects, supported drone skills, their estimated execution times, the required output format, and few-shot examples. The model is instructed to generate only subtasks that can be expressed using the predefined skill set. This constrains the LLM output and reduces the risk of hallucinated actions that cannot be executed by the system.

The output of this stage is represented as:

$$S = \{s_1, s_2, \ldots, s_n\},$$

where each subtask s_i is defined as:

$$s_i = (name_i, skill_i, object_i, service_i).$$

Here, $name_i$ is the subtask identifier, $skill_i$ is the required drone capability, $object_i$ is the target object in the environment, and $service_i$ is the estimated time required to execute the skill at the target object.

3.2.1 Example

Given the user-defined mission instruction `"Inspect Tower and measure wind conditions at House2."`, together with the available skills and objects defined in the world configuration:

```
skills = {
    'CaptureRGBImage': 1.1,
    'CaptureThermalImage': 2.4,
    'InspectStructure': 2.0,
    'MeasureWind': 1.5,
}

objects = {
    'House1': (4, 15, 39),
    'House2': (84, 37, 90),
    'SolarPanel': (72, 8, 84),
    'Tower': (53, 91, 57),
}
```

the decomposer generates the following atomic subtasks:

```
subtasks = [
    {
        "name": "SubTask1",
        "skill": "InspectStructure",
        "object": "Tower",
        "service_time": 2.0,
    },
    {
        "name": "SubTask2",
        "skill": "MeasureWind",
        "object": "House2",
        "service_time": 1.5,
    },
]
```

3.3 Task Allocation

The task allocator identifies which drones are capable of executing each decomposed subtask, creating a list of candidate drones whose skill set includes the skill required by the subtask. The allocator determines which assignments are possible, while the scheduler later selects the final drone for each subtask based on temporal criteria such as flight time and availability.

Each drone is described by its available skills, current position, and speed. In the allocation stage, only skill information is used. A drone qualifies for a subtask if the required skill of the subtask is included in the drone's skill list. Position and speed are intentionally ignored at this stage, since they are considered later by the scheduler when evaluating timing and efficiency.

Formally, let $D = \{d_1, d_2, \ldots, d_m\}$ denote the set of drones, and let K_j be the set of skills available to drone d_j. For a subtask s_i requiring skill $skill_i$, the candidate drone set is defined as:

$$C_i = \{d_j \in D \mid skill_i \in K_j\}.$$

If no drone satisfies the required skill, then $C_i = \emptyset$, indicating that the subtask is infeasible with the current drone team.

The allocator is implemented using few-shot prompting. The prompt provides the LLM with the required input format, allocation constraints, and examples of valid allocations. The model is instructed to output only structured Python code in the form of a list of dictionaries. Each dictionary preserves the original subtask information and adds a list of candidate drones capable of performing the task.

3.3.1 Example

Using the decomposed subtasks from Section 3.2.1 and the following drone configuration:

```
drones = {
    "Drone1":
    {
        "skills": ["InspectStructure", "CaptureImage"],
        "pos": (0, 0, 0),
        "speed": 1.0
    },
    "Drone2":
    {
        "skills": ["MeasureWind", "CaptureImage"],
        "pos": (1, 0, 0),
        "speed": 1.0
    },
    "Drone3":
    {
        "skills": ["InspectStructure", "MeasureWind"],
        "pos": (0, 1, 0),
        "speed": 1.0
    }
}
```

`SubTask1` requires the `InspectStructure` skill. Therefore, `Drone1` and `Drone3` are included in its candidate drone set. Similarly, `SubTask2` requires the `MeasureWind` skill, which can be performed by `Drone2` and `Drone3`. At this stage, the allocator does not select the final executing drone; it only identifies the feasible candidates for each subtask. The resulting allocated subtasks are:

```
subtasks_with_drones = [
    {
        "name": "SubTask1",
        "skill": "InspectStructure",
        "object": "Tower",
        "service_time": 2.0,
        "drones": ["Drone1", "Drone3"]
    },
    {
        "name": "SubTask2",
        "skill": "MeasureWind",
        "object": "House2",
        "service_time": 1.5,
        "drones": ["Drone2", "Drone3"]
    }
]
```

3.4 Task Scheduling

The scheduler converts the allocated subtasks into an explicit multi-drone execution schedule. While task decomposition determines what must be done and task allocation determines which drones can perform each task, scheduling assigns the final drone to each subtask and determines when each task should be executed.

The scheduler receives two inputs: the list of subtasks with their candidate drones from the task allocator, and a travel-time table describing the estimated travel time from each drone's initial position to each object, as well as between pairs of objects. The travel-time table is precomputed before scheduling in order to reduce the arithmetic burden on the LLM and ensure consistent timing estimates. This allows the LLM to focus on the combinatorial scheduling decision, namely selecting drones, ordering tasks, and minimizing the overall makespan, rather than repeatedly computing distances and travel times during generation.

The scheduler is constrained to assign each subtask exactly once across the drone team. Furthermore, a subtask may only be assigned to a drone contained in its candidate drone list. For each scheduled task, three temporal attributes are produced: *departure_time*, *arrival_time*, and *finish_time*. The departure time denotes when a drone starts flying towards the target object, the arrival time denotes when it reaches the object, and the finish time denotes when the required service has been completed. For a first task assigned to a drone, the arrival time is computed using the travel time from the drone's initial position to the task object. For consecutive tasks assigned to the same drone, the arrival time is computed using the travel time from the previous task object to the next task object. The finish time is then given by the arrival time plus the service time of the task.

Formally, for a task s_i assigned to drone d_j, the scheduled task is represented as:

$$\tau_i = (s_i, d_j, t_i^{dep}, t_i^{arr}, t_i^{fin}).$$

If s_i is the first task assigned to drone d_j, then:

$$t_i^{arr} - t_i^{dep} = T_{j,o_i}^{start},$$

where T_{j,o_i}^{start} is the travel time from drone d_j's initial position to object o_i. If s_i follows another task s_k on the same drone, then:

$$t_i^{arr} - t_i^{dep} = T_{j,o_k,o_i}^{obj},$$

where T_{j,o_k,o_i}^{obj} is the travel time for drone d_j from object o_k to object o_i. The finish time is defined as:

$$t_i^{fin} = t_i^{arr} + service_i.$$

The scheduler must also ensure that tasks assigned to the same drone do not overlap. For two consecutive tasks s_k and s_i assigned to the same drone, this requires:

$$t_k^{fin} \leq t_i^{dep}.$$

The scheduling objective is to minimize the total mission completion time, also known as the makespan:

$$\min \max_i t_i^{fin}.$$

This objective encourages the scheduler to exploit parallelism between drones and avoid unnecessary idle time.

The output of the scheduler is a Python dictionary indexed by drone name. Each drone is assigned an ordered list of tasks, where every task contains the target object, required skill, departure time, arrival time, and finish time. This explicit representation makes the generated plan directly inspectable and suitable for execution by the drone processes.

3.4.1 Example

Continuing the same example, the scheduler receives the allocated subtasks together with a precomputed travel-time table:

```
travel_times = {
    "drone_to_object": {
        "Drone1": {"Tower": 4.0, "House2": 6.0},
        "Drone2": {"Tower": 5.0, "House2": 3.0},
        "Drone3": {"Tower": 4.5, "House2": 4.0}
    },
    "drone_object_to_object": {
        "Drone1": {
            "Tower": {"House2": 2.5},
            "House2": {"Tower": 2.5}
        },
        "Drone2": {
            "Tower": {"House2": 2.0},
            "House2": {"Tower": 2.0}
        },
        "Drone3": {
            "Tower": {"House2": 1.5},
            "House2": {"Tower": 1.5}
        }
    }
}
```

One feasible schedule produced by the scheduler is:

```
schedule = {
    "Drone1": [
        {
            "name": "SubTask1",
            "object": "Tower",
            "skill": "InspectStructure",
            "departure_time": 0.0,
            "arrival_time": 4.0,
            "finish_time": 6.0
        }
    ],
    "Drone2": [
        {
            "name": "SubTask2",
            "object": "House2",
            "skill": "MeasureWind",
            "departure_time": 0.0,
            "arrival_time": 3.0,
            "finish_time": 4.5
        }
    ],
    "Drone3": []
}
```

In this schedule, `SubTask1` is assigned to `Drone1`. Since it is the first task assigned to `Drone1`, the arrival time is computed from the initial drone-to-object travel time:

```
travel_times["drone_to_object"]["Drone1"]["Tower"] = 4.0
```

With an inspection service time of 2.0 minutes, the finish time is $4.0 + 2.0 = 6.0$.

Similarly, `SubTask2` is assigned to `Drone2`. The travel time from `Drone2`'s initial position to `House2` is 3.0 minutes. With a service time of 1.5 minutes, the finish time is $3.0 + 1.5 = 4.5$.

The resulting makespan is therefore:

$$\max(6.0, 4.5) = 6.0.$$

3.5 Replanning

The system supports replanning when the original schedule can no longer be executed as intended. In real drone missions, this may occur due to changes in the drone state, low battery level, poor communication quality, or task requirements that exceed the drone's remaining capabilities. To detect such cases, the system uses a local LLM-based task admission module before executing a scheduled task and periodically during flight.

The local task admission module receives telemetry information from the drone and information about the planned task. The telemetry includes the maximum flight time, current battery percentage, battery health, link quality, drone state, estimated flight duration, and estimated task duration. Based on this information, the module classifies the scenario into one of three categories: `ok`, `drone_failure`, or `task_failure`. The `ok` decision indicates that the task can be executed as planned. The `drone_failure` decision indicates that the drone is currently not suitable for execution, for example due to poor connection, an emergency state, or insufficient operational readiness. The `task_failure` decision indicates that the task requirements exceed the drone's available resources, for example when the estimated mission duration is longer than the available flight time estimated by the model.

The task admission policy is provided to the LLM through a structured system prompt. The model is instructed to follow the policy and return a JSON object containing both the decision and a short explanation. The output is constrained to the following schema:

```
{
    "decision": "ok | drone_failure | task_failure",
    "reason": "<short explanation>"
}
```

The module evaluates the following conditions:

1. The drone is rejected with `drone_failure` if it is in an invalid state, such as `CONNECTING`, `EMERGENCY`, or `DISCONNECTED`.
2. The drone is rejected with `drone_failure` if the link quality is below the required threshold.
3. The task is rejected with `task_failure` if the required mission time exceeds the available flight time.

The available flight time is estimated from the maximum flight time, battery percentage, and battery health:

$$T_{\text{available}} = T_{\max} \cdot \frac{B_{\text{perc}}}{100} \cdot \frac{B_{\text{health}}}{100}.$$

The required mission time is computed as:

$$T_{\text{required}} = T_{\text{flight}} + T_{\text{task}}.$$

If $T_{\text{available}} < T_{\text{required}}$, the task is rejected with `task_failure`. Otherwise, no rejection rule is triggered and the task is accepted.

When the local admission module returns `ok`, the drone proceeds with the scheduled task. When it returns `drone_failure`, the planner removes the affected drone from the set of currently available drones and regenerates the schedule for the unfinished subtasks. When it returns `task_failure`, the planner treats the current assignment as infeasible and attempts to reassign the affected task to another capable drone. In both failure cases, completed tasks are preserved, while unfinished tasks are returned to the planning pipeline.

In the current implementation, a subtask is marked as completed only after the drone reports successful completion of the full assigned action. If a drone fails before this completion message is received, the subtask is treated as unfinished and is returned to the planning pipeline as a full subtask. Partial progress within a subtask is not credited, since the system does not yet model partial completion of sensing or inspection actions.

Although the current implementation evaluates explicit conditions for drone state, link quality, and available flight time, the use of an LLM-based task admission module is motivated mainly by extensibility. In the current system, these conditions could also be implemented using conventional rule-based logic. However, an LLM-based module allows the admission policy to be adapted through natural-language prompt changes as new platforms, telemetry fields, and mission-specific constraints are introduced.

This is useful in future deployments where feasibility decisions may depend on more ambiguous information than the numerical fields used in the simulation. For example, different drone platforms may report state, communication quality, battery warnings, or payload limitations using different terminology or diagnostic messages. An LLM can help interpret such platform-specific descriptions and map them to a common admission decision, such as `ok`, `drone_failure`, or `task_failure`. Additional policies, such as weather restrictions, safety margins, no-fly-zone constraints, payload-dependent battery consumption, or task-specific risk levels, can also be incorporated by modifying the prompt and output schema rather than rewriting the admission logic.

Therefore, the LLM-based admission module should be viewed as a flexible policy interface for execution-time feasibility checking. In this thesis, it is evaluated on explicit conditions to ensure controlled and measurable behavior, while its main advantage is the ability to extend the same mechanism to richer telemetry descriptions, platform-specific policies, and more complex mission constraints in future work.

3.6 System Architecture

This section describes the architecture of the implemented multi-drone planning and replanning system.

3.6.1 Overview

The architecture consists of four main parts:

- a planner process responsible for task decomposition, allocation, scheduling, execution monitoring and replanning;
- multiple drone worker processes each representing one drone;
- a world configuration defining objects, skills, and drone capabilities;

- a queue-based communication system connecting the planner and the drones.

An overview of the implemented architecture is shown in Figure 3.2.

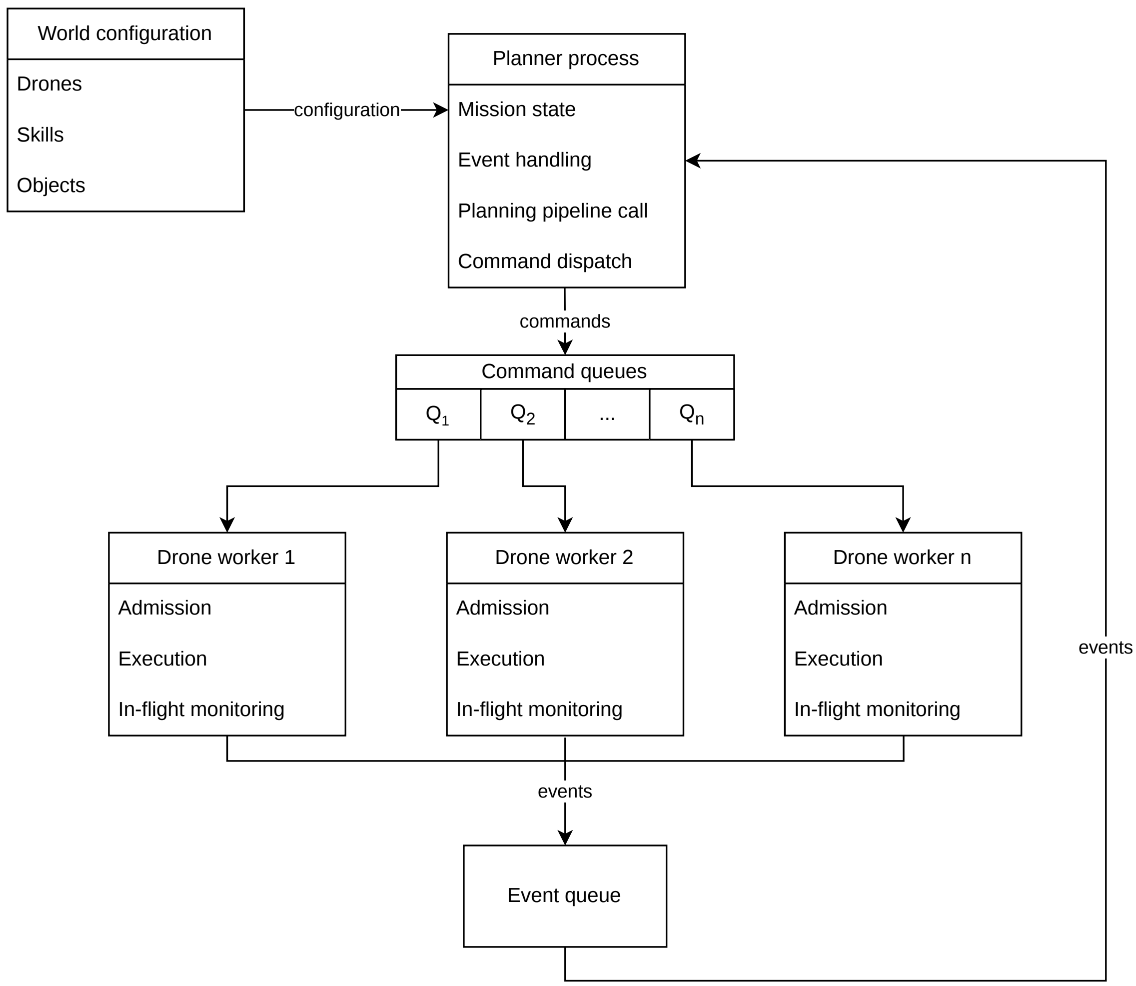


Figure 3.2: Software architecture of the implemented multi-drone planning and replanning system. The planner process receives the world configuration, calls the planning pipeline, dispatches commands through per-drone command queues, and receives execution events through a shared event queue.

3.6.2 System Components

Planner

The planner is the central coordination component of the system. It receives the mission instructions, calls the language models, assigns subtasks to drones and monitors the mission during execution.

The planner maintains the current mission state, including which subtasks are pending, assigned, completed, rejected or failed. Based on this state, it decides whether normal execution should continue or whether replanning is required. If a drone rejects a task or fails during execution, the planner updates the mission state and attempts to reallocate the remaining work to other available drones.

The planner is also responsible for translating symbolic task assignments into concrete command messages. These commands are sent to the corresponding drone worker through a dedicated command queue.

Drone Worker Processes

Each drone is represented by a separate worker process. A drone worker receives commands from the planner, checks whether the requested skill can be executed, performs the action, and reports the result back to the planner.

The system supports switching between simulation and real-world modes. In simulation mode, the worker does not communicate with a physical drone. Instead, it is linked with a visualization tool that shows task allocations and drone movement. In real-world mode, the worker sends commands to the actual drone interface.

This process-based design isolates drone execution from the planner. If one drone fails or rejects a task, the planner can handle the event without stopping the entire system.

World Configuration

The world configuration defines the information required by the planner and drone workers. This includes the available drones, with their skills, starting positions, speeds, flight altitudes, and maximum flight times, as well as the positions of objects in the environment.

Communication Architecture

Communication between the planner and drone workers is implemented using multiprocessing queues. The system uses one shared event queue and one command queue per drone.

The command queues are used by the planner to send commands to specific drones. Each drone worker listens only to its own command queue. The shared event queue is used by all drone workers to report execution events back to the planner.

The communication structure can be summarized as follows:

- `command_queues`: a dictionary of per-drone queues used to send commands from the planner to drone workers;
- `event_queue`: a shared queue used by drone workers to send status updates back to the planner;
- drone worker processes: independent processes that execute commands and return events;

- planner process: the central process that dispatches commands and reacts to events.

This architecture supports asynchronous execution. Multiple drones can receive and execute commands independently, while the planner continuously monitors the returned events.

Execution Flow

A mission begins with a natural-language instruction provided by the user. The planner sends this instruction, together with information about the available drones, skills, and environment, to the LLM-based planning pipeline. The pipeline returns a schedule containing the generated subtasks, their assigned drones, and their planned execution order.

Each drone is first assigned its initial subtask from the generated schedule. The local task admission module then decides whether to accept or reject the proposed subtask. Execution only begins once all drones have accepted their assigned proposals. If any drone rejects its proposed subtask, the schedule is recomputed before execution begins.

Once execution has started, the same module periodically monitors the drone state and reports failures when detected. If no failures are reported, the drones continue according to the original schedule. If a drone fails during execution, the schedule is recomputed for the remaining subtasks that have not yet been completed or started. This replanning step also considers the expected completion times of busy drones, so that drones already executing subtasks can be included in the updated schedule once they become available.

This execution loop continues until all subtasks are completed or no feasible assignment can be found.

Chapter 4

Results

This chapter presents the evaluation results of the proposed LLM-based drone planning and replanning architecture. The evaluation is divided into three parts. First, the individual LLM-based components are evaluated. Second, the selected components are integrated and evaluated in an end-to-end simulation environment across multiple mission scenarios, followed by a representative case study. Finally, the physical drone platform and laboratory setup are described to relate the proposed architecture to a real-world deployment environment.

4.1 Component-Level LLM Evaluation

This section evaluates the two LLM-based components used in the proposed architecture: the cloud-based planner and the task admission module. The purpose of this evaluation is to select the models used in the subsequent simulation and physical integration setup.

4.1.1 Cloud-based Planning Pipeline

The cloud-based planner was evaluated to determine which model provided the best trade-off between schedule quality, inference time, and reliability. Both cloud-based and locally deployable models were initially considered. However, due to hardware constraints and latency requirements, small quantized LLMs with 1–3 billion parameters were found to be too slow and unreliable for planning, even for tasks consisting of only one or two subtasks. The final cloud-based evaluation therefore used GPT-5, GPT-5-mini, GPT-4o, and GPT-4.1 through the OpenAI API [26].

Evaluation Setup

The models were tested on tasks containing between one and ten subtasks. For each subtask count, three task descriptions were evaluated. The task descriptions were written in unstructured natural language, using varied wording and sentence structure in order to test the models' ability to interpret high-level mission instructions.

All planning cases were generated using the same world configuration, consisting of a heterogeneous drone team and a set of predefined target objects. The drone team contained agents with different skill sets, so not every drone was capable of executing every subtask. The planning dataset contained 30 mission descriptions, with three missions for each subtask count from one to ten. In total, the dataset contained 165 subtasks.

The evaluated mission descriptions varied the requested target objects and required skills. As summarized in Table 4.1, the dataset covered five skill types: RGB image capture, thermal image capture, video recording, structure inspection, and wind measurement. The subtasks were distributed across nine target objects, including houses, rooftops, solar panels, the tower, and the base. The planner was therefore evaluated on missions requiring different sensing capabilities and target locations, and was required to decompose each mission, identify suitable drones, and generate a valid schedule for the resulting subtasks.

Table 4.1: Distribution of skills and target objects in the planning evaluation dataset.

Skill	Count	Object	Count
CaptureRGBImage	45	RoofTop1	26
CaptureThermalImage	42	House1	23
InspectStructure	29	RoofTop2	20
RecordVideo	27	House2	19
MeasureWind	22	Tower	18
		House3	18
		SolarPanel1	15
		SolarPanel2	14
		Base	12

The generated schedules were compared against an optimal schedule computed using a VRP solver [27]. The VRP solution was used as a baseline for evaluating the makespan of the LLM-generated mission schedules. A generated schedule was considered successful if it produced a valid sequence of drone actions satisfying all subtasks.

Schedule Quality

Table 4.2 summarizes the overall validity rate of each planning model together with its average makespan gap from the VRP baseline computed over valid schedules, while Table 4.3 gives a more detailed breakdown by mission complexity, showing the makespan gap for each subtask count and the number of valid schedules out of three test cases.

Table 4.2: Overall planning validity and average makespan difference from the VRP baseline. The average gap is computed only over valid schedules.

Model	Valid schedules	Validity rate	Avg. VRP gap
GPT-5	30/30	100.0%	0.49%
GPT-5-mini	28/30	93.3%	5.93%
GPT-4o	23/30	76.7%	60.12%
GPT-4.1	19/30	63.3%	64.91%

Table 4.3: Average makespan difference between the planning pipeline and the VRP baseline by number of subtasks. Values in parentheses indicate the number of solved cases out of three tasks. Averages are computed over valid schedules only. Cases with no valid solutions are indicated by –.

Model Subtasks	GPT-5	GPT-5-mini	GPT-4o	GPT-4.1
1	0.00 (3/3)	0.00 (3/3)	0.00 (3/3)	0.00 (3/3)
2	0.00 (3/3)	0.00 (3/3)	14.07 (3/3)	0.00 (3/3)
3	0.00 (3/3)	0.00 (3/3)	36.54 (3/3)	62.60 (3/3)
4	0.00 (3/3)	0.00 (3/3)	121.29 (3/3)	121.29 (3/3)
5	0.00 (3/3)	0.00 (3/3)	132.79 (3/3)	112.17 (3/3)
6	0.00 (3/3)	0.86 (3/3)	78.51 (3/3)	–
7	0.00 (3/3)	5.77 (2/3)	62.95 (3/3)	52.03 (2/3)
8	0.00 (3/3)	14.90 (3/3)	–	–
9	0.56 (3/3)	15.88 (2/3)	22.16 (2/3)	79.66 (1/3)
10	4.35 (3/3)	25.19 (3/3)	–	161.43 (1/3)

Inference Time

Table 4.4 shows the average inference time for each model, computed only over successfully generated schedules.

Table 4.4: Average inference time of the planning pipeline by number of subtasks. Values in parentheses indicate solved cases over three tasks. Averages are computed over valid schedules only. Cases with no valid solutions are indicated by –.

Model Subtasks	GPT-5	GPT-5-mini	GPT-4o	GPT-4.1
1	32.97 (3/3)	20.27 (3/3)	6.17 (3/3)	4.07 (3/3)
2	49.90 (3/3)	21.87 (3/3)	5.00 (3/3)	4.03 (3/3)
3	94.50 (3/3)	33.50 (3/3)	8.93 (3/3)	7.13 (3/3)
4	111.90 (3/3)	51.63 (3/3)	12.90 (3/3)	7.13 (3/3)
5	112.33 (3/3)	45.83 (3/3)	15.63 (3/3)	11.30 (3/3)
6	205.07 (3/3)	98.10 (3/3)	14.47 (3/3)	–
7	284.37 (3/3)	119.30 (2/3)	17.90 (3/3)	17.00 (2/3)
8	284.77 (3/3)	133.67 (3/3)	–	–
9	243.07 (3/3)	112.05 (2/3)	19.70 (2/3)	16.50 (1/3)
10	269.00 (3/3)	97.73 (3/3)	–	16.80 (1/3)

Model Selection

The results show that GPT-5 produced valid schedules for all evaluated cases and remained closest to the VRP baseline, but it also required the longest inference time. GPT-5-mini achieved a slightly lower validity rate, with 28 valid schedules out of 30, but maintained near-optimal schedules for small and medium task sets while requiring substantially less inference time. In contrast, GPT-4o and GPT-4.1 were faster, but their validity rates decreased for larger missions. Considering validity rate, optimality gap, inference time, and cost together, GPT-5-mini was selected for the cloud-based planner.

4.1.2 Local Task Admission Module

The task admission module has different requirements than the cloud-based planner. While the planner must produce high-quality mission schedules, the admission module must evaluate drone telemetry quickly and reliably during execution. Since this module directly affects whether a task is accepted, rejected, or sent for replanning, reliability is particularly important.

Evaluation Setup

The evaluated models were tested on four categories: acceptable tasks, drone-state errors, link-quality errors, and flight-time errors. Each category contained 20 test cases, resulting in 80 test cases per model. Seven 4-bit quantized models, ranging from 0.6 to 4 billion parameters, were evaluated from the Qwen [28, 29], Gemma [30], Llama [31], and Phi [32] model families.

Classification Accuracy

Table 4.5 reports the number of correctly classified cases out of 20 for each category.

Table 4.5: Task admission accuracy across failure categories. Each value indicates the number of correctly classified cases out of 20.

Model	Acceptable task	Drone state error	Link quality error	Flight time error
`qwen3:0.6b`	18	8	6	16
`gemma3:1b`	20	15	0	0
`qwen3:1.7b`	20	20	20	20
`qwen2.5:3b`	7	20	20	9
`llama3.2:3b`	13	20	18	0
`phi4-mini:3.8b`	20	20	20	1
`gemma3:4b`	14	20	9	0

Inference Time

Table 4.6 reports the average and maximum inference time for each model.

Table 4.6: Average and maximum inference time for the task admission module.

Model	Average inference time (s)	Maximum inference time (s)
`qwen3:0.6b`	7.99	24.75
`gemma3:1b`	0.76	2.83
`qwen3:1.7b`	3.60	7.45
`qwen2.5:3b`	1.02	6.44
`llama3.2:3b`	1.25	14.24
`phi4-mini:3.8b`	1.52	7.72
`gemma3:4b`	1.20	9.12

Model Selection

The results show that `qwen3:1.7b` achieved perfect classification accuracy across all four categories, correctly solving all 80 test cases. Although some models achieved lower average inference times, they failed on safety-relevant categories such as link-quality or flight-time errors. Since the task admission module directly affects replanning decisions, classification reliability was prioritized over minimal inference time. Therefore, `qwen3:1.7b` was selected for the local task admission module.

4.1.3 Reproducibility Details

Because LLM outputs may vary across model versions, API updates, inference parameters, and runtime environments, the evaluation was documented with the exact model identifiers and execution settings used during testing.

Table 4.7 reports the shared hardware and software environment. Table 4.8 provides the runtime settings used for the local task admission models, while Table 4.9 reports the corresponding settings for the cloud-based planning models. The exact model identifiers are reported in Tables 4.10 and 4.11 for the local and cloud-based models, respectively.

Table 4.7: Shared reproducibility details for the LLM evaluations.

Item	Value
Python version	`3.12.10`
Operating system	Ubuntu 22.04.5 LTS
Hardware	Intel Core i5-11320H CPU 16 GB RAM NVIDIA RTX 3050 Laptop GPU, 4 GB VRAM

Table 4.8: Reproducibility details for local task admission model evaluation.

Item	Value
Evaluation date	02.04.2026
Inference framework	Ollama
Ollama version	`0.24.0`
Ollama Python package version	`0.6.1`

Inference settings	Greedy decoding
	`temperature = 0.0`
	`top_p = 1.0`
	`top_k = 1`

Table 4.9: Reproducibility details for cloud-based planning model evaluation.

Item	Value
Evaluation date	30.04.2026
API provider	OpenAI API
OpenAI Python package version	`2.26.0`
Inference settings	GPT-5 models: `temperature = 1.0`† GPT-4 models: `temperature = 0.0`

† For GPT-5 models, the OpenAI API only supports the default temperature value of `1.0`. For non-GPT-5 models, `temperature = 0.0` was explicitly set to reduce sampling variability.

Table 4.10: Local LLM model metadata used in the task admission evaluation.

Model tag	Model ID	Size	Parameters	Quantization
qwen3:0.6b	7df6b6e09427	522 MB	751.63M	Q4_K_M
qwen3:1.7b	8f68893c685c	1.4 GB	2.0B	Q4_K_M
qwen2.5:3b	357c53fb659c	1.9 GB	3.1B	Q4_K_M
llama3.2:3b	a80c4f17acd5	2.0 GB	3.2B	Q4_K_M
phi4-mini:3.8b	78fad5d182a7	2.5 GB	3.8B	Q4_K_M
gemma3:4b	a2af6cc3eb7f	3.3 GB	4.3B	Q4_K_M
gemma3:1b	8648f39daa8f	815 MB	999.89M	Q4_K_M

Table 4.11: Cloud model identifiers used in the planning pipeline evaluation.

Reported name	API model name	Snapshot identifier
GPT-5	gpt-5	gpt-5-2025-08-07
GPT-5-mini	gpt-5-mini	gpt-5-mini-2025-08-07
GPT-4o	gpt-4o	chatgpt-4o-latest
GPT-4.1	gpt-4.1	gpt-4.1-2025-04-14

4.2 Simulation Results

This section evaluates the planning and task admission components in a simulated multi-drone mission environment. The purpose of the simulation is to validate the interaction between cloud-based planning, local task admission, execution monitoring, failure detection, and replanning before deployment on a physical drone platform.

4.2.1 Simulation Setup

The simulation environment was designed to represent a multi-drone inspection scenario with static objects of interest distributed across a two-dimensional area.

The simulation used the same drone team and target objects as the cloud-based planning evaluation described in Section 4.1.1. The simulator abstracts away low-level flight dynamics and instead focuses on task-level execution, including task allocation, admission checking, failure handling, and replanning. For visualization, the simulated drone movements are projected onto a two-dimensional plane.

To evaluate the system response to execution-time changes, the simulator also models battery drainage, link-quality variations, and drone failures. Battery levels decrease during mission execution, while link quality may change over time and affect whether a drone can safely accept a task. Drone failures can occur during execution, causing the affected drone's remaining subtasks to be returned to the cloud-based planner for rescheduling. These simulated disturbances allow the replanning and task admission mechanisms to be evaluated under dynamic mission conditions.

4.2.2 Simulation Evaluation

To evaluate the robustness of the complete architecture, the simulation was executed over ten mission scenarios. Each scenario used a different mission description with varying target objects and increasing complexity in terms of the number of subtasks.

A mission was classified as *Completed* if all subtasks were completed without requiring replanning. It was classified as *Recovered* if at least one rejection or runtime failure occurred, but replanning produced a valid updated schedule and all subtasks were eventually completed. It was classified as *Infeasible* if the system correctly determined that no remaining drone could execute one or more unfinished subtasks. Infeasible missions were not counted as system failures, since the correct behavior in this situation is to stop rather than assign an infeasible task. Table 4.12 shows the simulation results across the ten mission scenarios.

Table 4.12: End-to-end simulation results across ten mission scenarios.

Mission	Subtasks	Outcome	Completed subtasks	Initial plan [s]	Replans	Replan reason	Subtasks replanned	Replan latency [s]
M1	1	Completed	1/1	14.92	0	–	–	–
M2	2	Completed	2/2	27.81	0	–	–	–
M3	3	Completed	3/3	26.82	0	–	–	–
M4	4	Recovered	4/4	53.70	2	Drone state error; link quality error	1, 1	6.07, 6.36
M5	5	Recovered	5/5	64.36	1	Low link quality	1	5.45
M6	6	Completed	6/6	59.83	0	–	–	–
M7	7	Recovered	7/7	95.64	1	Drone state error	1	6.22
M8	8	Recovered	8/8	96.26	1	Flight-time error	2	12.66
M9	9	Completed	9/9	161.76	0	–	–	–
M10	10	Infeasible	8/10	94.93	2	Drone state error; link quality error	5, 5	25.68, 29.18

Note: For missions with multiple replanning events, the replan reasons, subtasks replanned, and replanning latencies are reported in chronological order.

Across the ten simulation scenarios, nine missions were completed, either directly or after replanning, giving a mission completion rate of 90.0%. One mission was classified as infeasible because no available drone could safely execute the remaining subtasks. Since this condition was correctly detected by the system, the feasibility-aware success rate was 100.0%. Overall, 53 out of 55 subtasks were completed, corresponding to a task completion rate of 96.4%. The average initial planning latency was 12.66 s per subtask, while the average replanning latency was 5.73 s per remaining subtask. The lower normalized replanning latency is expected, since replanning is performed only for unfinished subtasks and reuses the existing decomposed and allocated subtask representation instead of repeating the LLM-based decomposition and allocation stages. No invalid system failures occurred.

4.2.3 Representative Simulation Case Study

To illustrate the end-to-end behavior of the system, this section presents a representative mission execution in which the drone team executed the following mission:

> Inspect both solar panels, record thermal image of the tower and both rooftops.

Based on the submitted natural-language mission, the cloud-based planner generated a schedule, and the resulting subtasks were sent to the drones for admission checking.

Table 4.13: Subtasks and suitable drones in the simulated mission case study.

Subtask	Object	Required skill	Suitable drones
SubTask1	SolarPanel1	InspectStructure	Drone5, Drone6
SubTask2	SolarPanel2	InspectStructure	Drone5, Drone6
SubTask3	Tower	CaptureThermalImage	Drone1, Drone3, Drone5
SubTask4	RoofTop1	CaptureThermalImage	Drone1, Drone3, Drone5
SubTask5	RoofTop2	CaptureThermalImage	Drone1, Drone3, Drone5

Table 4.14: Initial schedule generated for the simulated mission case study.

Drone	Subtask	Target object	Travel interval [min]	Execution interval [min]
Drone1	SubTask3	Tower	0.0–8.0	8.0–10.4
Drone3	SubTask4	RoofTop1	0.0–7.1	7.1–9.5
Drone5	SubTask5	RoofTop2	0.0–2.9	2.9–5.3
	SubTask2	SolarPanel2	5.3–9.9	9.9–11.2
Drone6	SubTask1	SolarPanel1	0.0–4.6	4.6–5.9
			Makespan	**11.2 min**

Table 4.15: Updated schedule generated for the simulated mission case study.

Drone	Subtask	Target object	Travel interval [min]	Execution interval [min]
Drone1	SubTask4	RoofTop1	0.0–14.0	14.0–16.4
Drone3	SubTask3	Tower	0.0–2.2	2.2–4.6
Drone5	SubTask5	RoofTop2	0.0–2.9	2.9–5.3
	SubTask2	SolarPanel2	5.3–9.9	9.9–11.2
Drone6	SubTask1	SolarPanel1	0.0–4.6	4.6–5.9
			Makespan	**16.4 min**

The main steps of the simulation are as follows:

1. The planning pipeline first decomposes the mission into subtasks and then identifies the capable drones for each subtask during allocation (Table 4.13).
2. The planning pipeline creates an initial schedule with a makespan of 11.2 min (Table 4.14), and each drone's task admission module receives its first subtask (Figure 4.1a).
3. Drone3 rejects its assigned subtask, SubTask4, with the following message:

   ```
   available flight time (7.61 min) < required mission time (9.5 min)
   ```

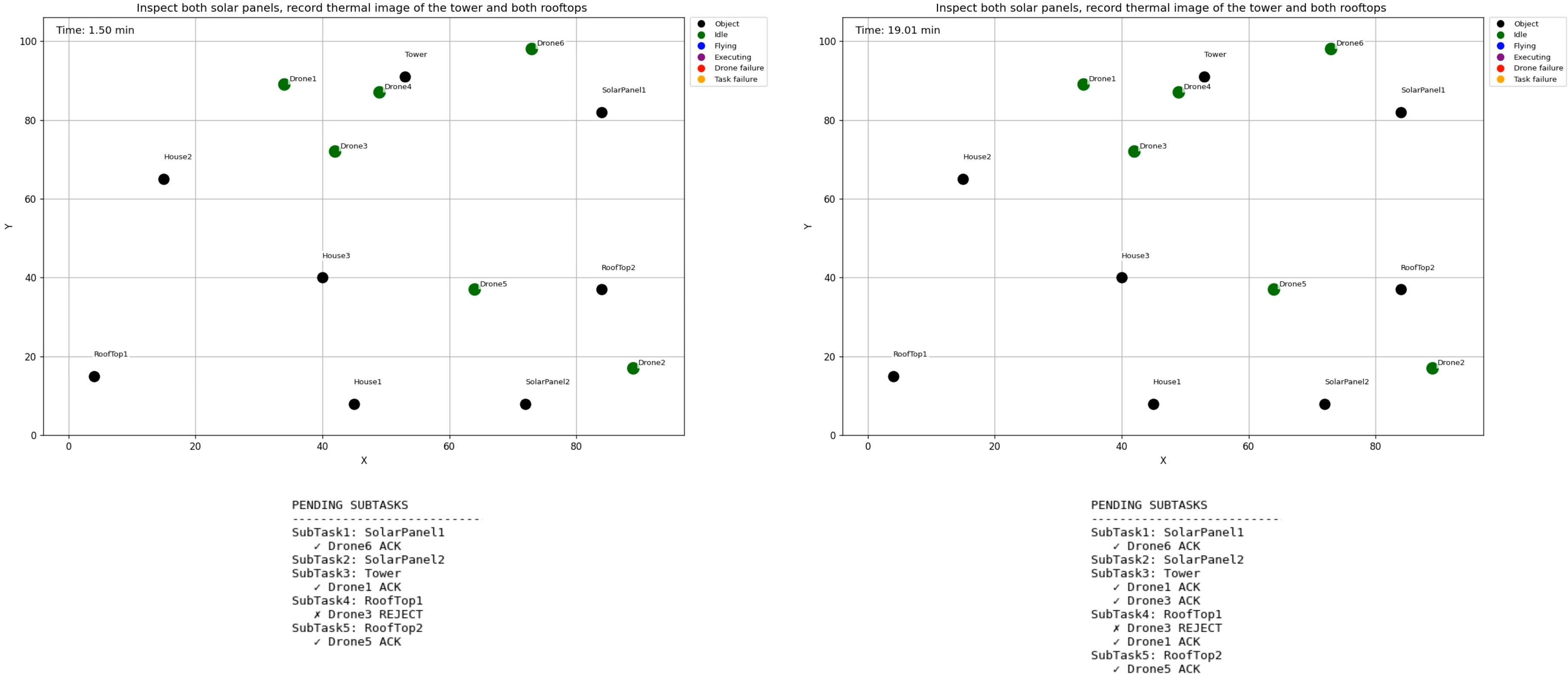
4. The planner removes Drone3 as a candidate for SubTask4 and generates an updated schedule with a longer makespan of 16.4 min (Table 4.15).
5. The updated schedule is acknowledged by all drones (Figure 4.1b).

6. The drones start flying towards their target objects (Figure 4.2a) and begin execution once they arrive (Figure 4.2b).
7. After completing their assigned subtasks, the drones become idle (Figure 4.3a).
8. Since no issues were detected during the previous subtasks, no rescheduling is required. Drone5 therefore receives its next pending subtask from the existing schedule.
9. Drone5's onboard admission module accepts the subtask, after which it starts flying towards SolarPanel2 (Figure 4.3b).
10. During execution at RoofTop1, Drone1 performs a periodic in-flight admission check using the task admission module (Figure 4.4a), which returns:

    ```
    Drone is in suitable state, link quality 5,
    available flight time > required mission time
    ```

    This indicates that the execution can continue.
11. Drone1 and Drone5 finish their subtasks, thereby completing the user-defined mission (Figure 4.4b).

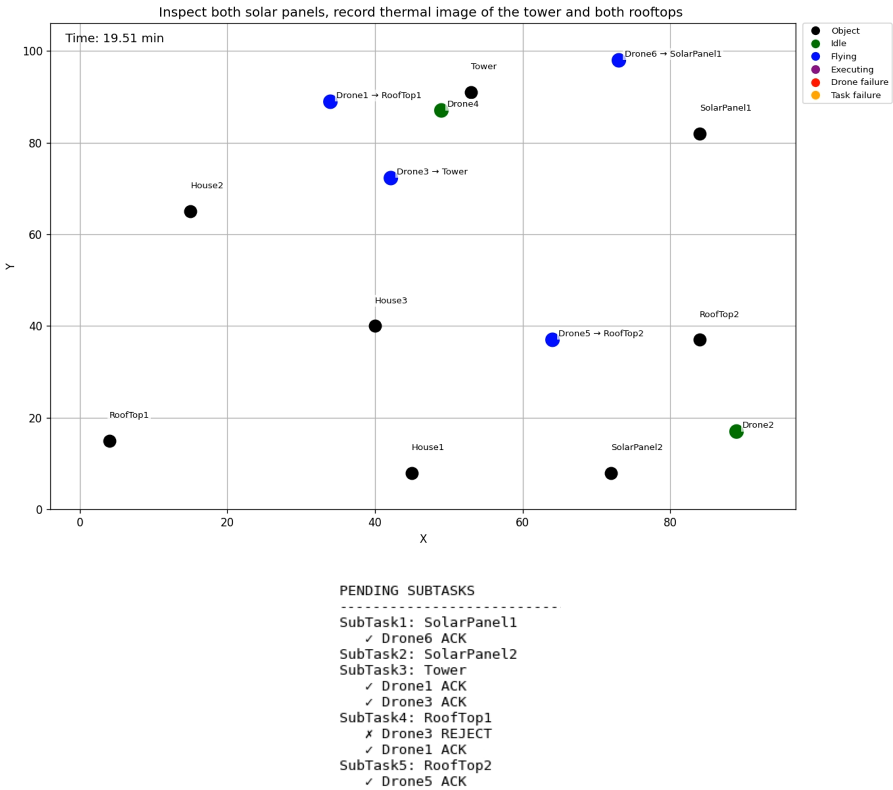
This qualitative case study demonstrates that the implemented architecture can coordinate the cloud-based planner, local task admission module, and execution monitor within a single mission workflow. The sequence shows that the system can identify an infeasible task assignment during local admission checking, update the allocation by excluding the unsuitable drone for the affected subtask, generate an updated schedule, and continue the mission with the remaining suitable drones.



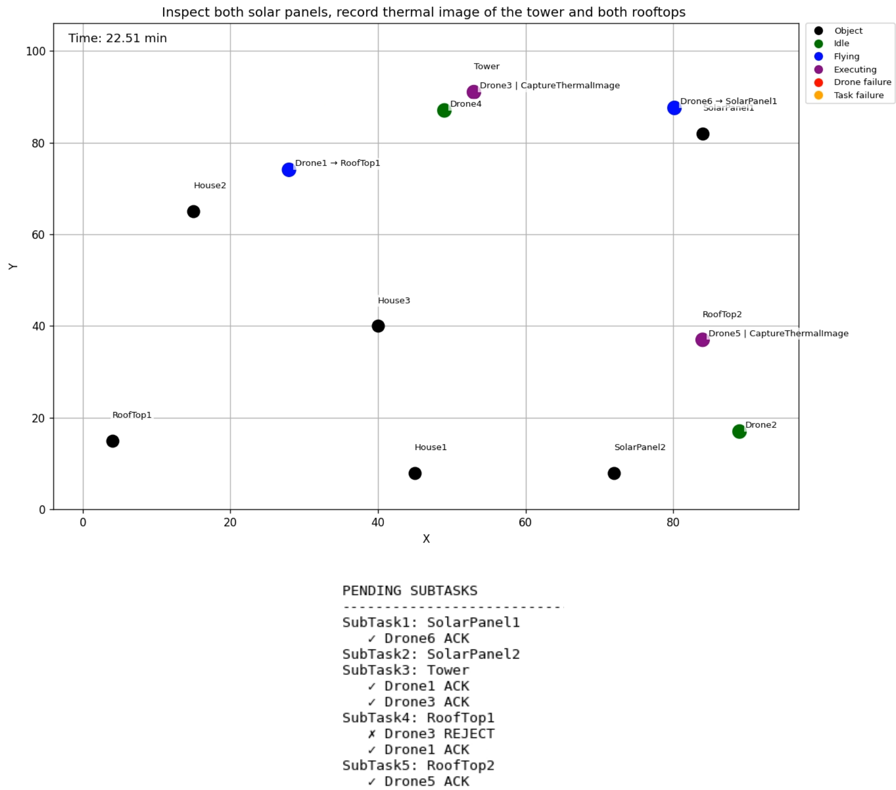
(a) First allocation round with rejection from Drone3.

(b) Second, successful allocation round.

Figure 4.1: Initial allocation attempts in the simulated mission.

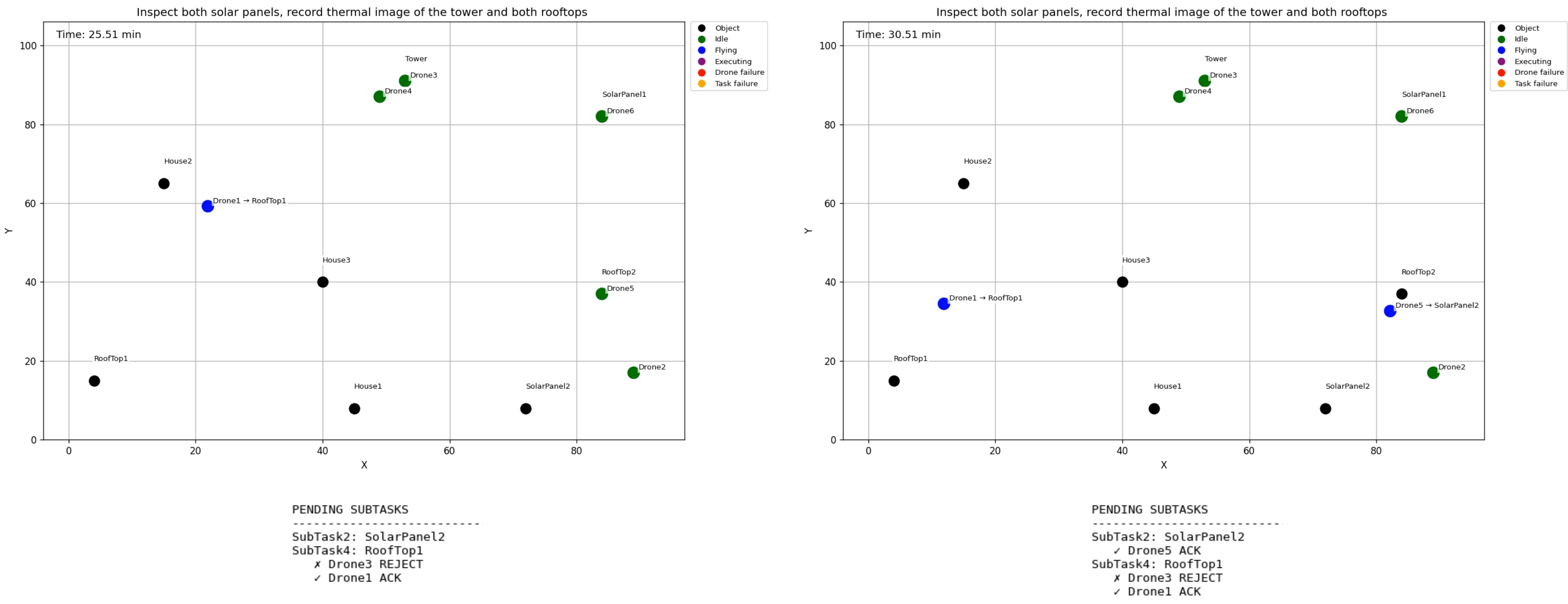


(a) The drones move towards their target objects.



(b) Arrived drones start executing their subtasks.

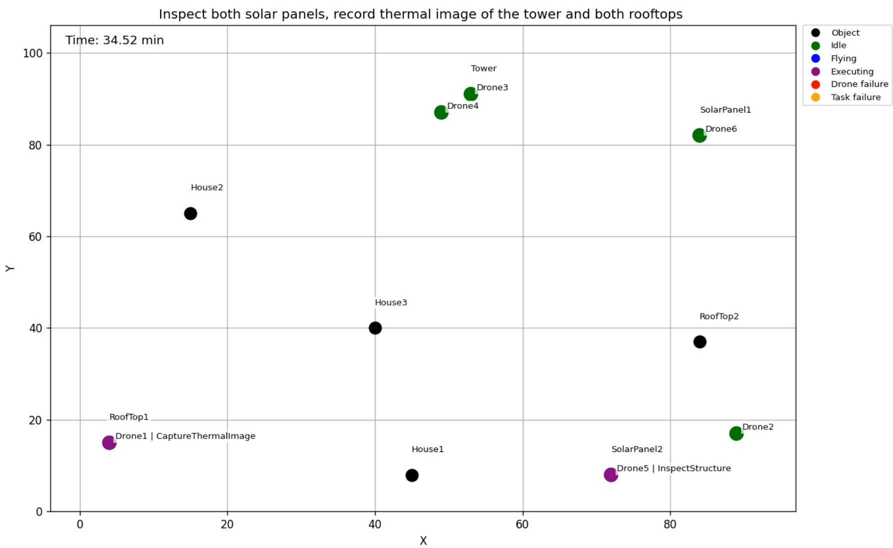
Figure 4.2: Drone navigation and subtask execution during the simulated mission.



(a) Drones become idle after completing their subtasks.

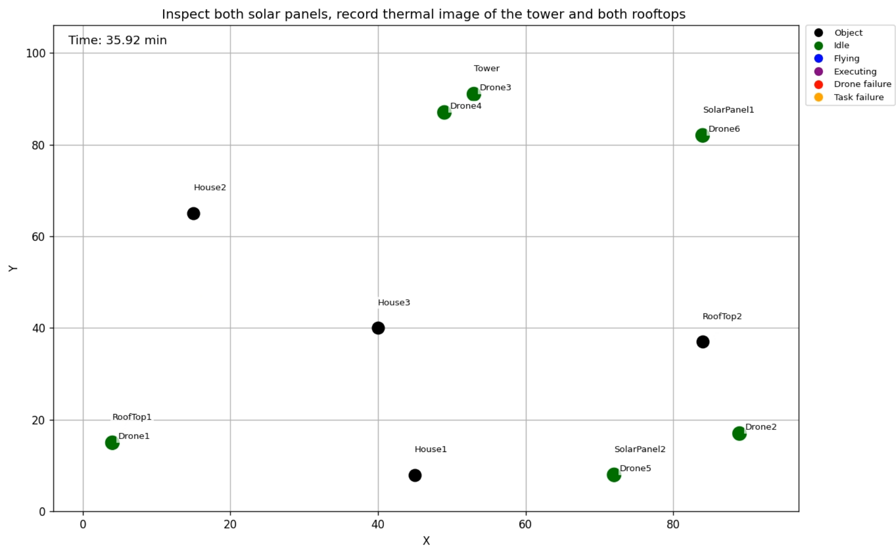
(b) Drone5 accepts its next task and starts moving towards the target object.

Figure 4.3: Transition between completed and newly assigned subtasks.




```
PENDING SUBTASKS
-------------------------
SubTask2: SolarPanel2
   ✓ Drone5 ACK
SubTask4: RoofTop1
   ✗ Drone3 REJECT
   ✓ Drone1 ACK
   ✓ Drone1 IN-FLIGHT OK
```

(a) Drone1 performs an in-flight task admission check.




```
PENDING SUBTASKS
-------------------------
All subtasks completed
```

(b) All subtasks are completed.

Figure 4.4: In-flight task admission checking and mission completion.

4.3 Physical Integration Readiness

This section describes the physical drone platform, laboratory environment, and control interface prepared for future real-world deployment of the proposed architecture. The purpose of this section is not to report a physical validation experiment, but to document the readiness of the system for future physical integration.

The full closed-loop architecture, including cloud-based planning, local LLM-based task admission, and replanning, was not deployed on the physical drones within the scope of this thesis. Therefore, the complete system behavior is evaluated in simulation. The physical setup is included to show how the symbolic planner could be connected to a real drone platform and control interface.

4.3.1 Drone Platform

For future physical integration, drones from the Parrot Anafi family were considered as the target platform. To represent a drone team with heterogeneous sensing capabilities, four variants were considered: the Parrot Anafi 4K, Anafi Thermal, Anafi USA, and Anafi Ai. These platforms are shown in Figure 4.5.

(a) Anafi 4K. **(b)** Anafi Thermal.

(c) Anafi USA. **(d)** Anafi Ai.

Figure 4.5: Drone platforms considered for physical testing. Images adapted from Parrot product documentation [33–36].

Table 4.16 summarizes the relevant specifications of the Anafi drone variants considered for future physical integration, together with the symbolic skills that could be assigned to each platform.

Table 4.16: Capabilities of the Anafi drone variants considered for physical integration. Specifications adapted from [37].

Capability	Anafi 4K	Anafi Thermal	Anafi USA	Anafi Ai
Max. flight time [min]	25	26	32	32
Max. horizontal speed [m/s]	15	15	14.7	16
RGB camera [MP]	21	21	21	48
Video capability	4K/FHD	4K/FHD/HD	4K/FHD/HD	4K/FHD
Thermal sensing	No	Yes	Yes	No
Zoom [x]	1–3	1–3	1–32	1–6
Assigned skills	RGB image Video Zoom image	RGB image Thermal image Zoom image Video Dual-spectral	RGB image Thermal image Zoom image Video Dual-spectral	RGB image Video Zoom image

Note: Dual-spectral refers to combined RGB and thermal inspection.

4.3.2 Laboratory and Control Setup

The intended physical setup consists of an indoor laboratory environment, a Vicon-based localization system, and a ROS-based control interface for communicating with the Anafi drones. This setup provides the infrastructure required for executing high-level mission commands on the physical drone platform.

Laboratory Environment

The laboratory environment is equipped with a Vicon motion capture system [38]. Since GPS-based localization is not available indoors, the Vicon system can provide position and orientation feedback during flight. Reflective markers can be mounted on the drone, allowing the motion capture system to track the drone within the available flight area.

Similar target objects to those used in the simulation environment can be defined in the physical setup. Unlike the two-dimensional simulation setup, the laboratory environment is represented in three dimensions. Each target object can be assigned a position in the Vicon coordinate frame, allowing the drone to navigate to target locations using 3D position feedback.

Drone Control Interface

The physical setup uses the `anafi_autonomy` ROS package [37] as the control interface between the mission execution pipeline and the Parrot Anafi drones. The package provides ROS topics and services for accessing sensor data, controlling the drones through high-level mission commands such as takeoff, landing, and navigation, and obtaining the drone-state information required by the task admission module, such as battery level, flight state, position feedback, and communication status.

The LLM-based planner does not directly control the drone hardware. Instead, it generates symbolic subtasks, which are translated by the mission execution layer into commands supported by the drone control interface. This separation allows the planning system to remain platform-independent, while the execution layer handles platform-specific communication with the Anafi drones.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

The main contribution of this thesis is not only the use of LLMs for task planning, but the combination of four key elements in one multi-drone architecture: natural-language mission interpretation, capability-aware allocation, explicit schedule generation, and local task admission with replanning of unfinished subtasks. This distinguishes the proposed system from LLM-based robotic planning approaches that generate sequential plans, dialogue-based coordination, or reactive updates without explicit per-drone schedules and execution-time admission checking.

A cloud-based planning pipeline was developed in which a user mission is transformed into atomic subtasks, assigned to suitable drones based on their capabilities, and scheduled while considering travel time and task duration. The resulting output is an explicit per-drone schedule containing departure, arrival, and finish times. In addition, a local task admission module was developed to evaluate whether individual drones are able to accept assigned tasks based on their state, link quality, and available flight time.

The system was evaluated in three parts. First, several cloud-based LLMs were compared on missions containing one to ten subtasks and evaluated against a vehicle routing problem baseline. The results showed that GPT-5-mini provided the best trade-off between schedule quality, validity, inference time, and cost, and was therefore selected for the cloud-based planner. Second, seven quantized locally deployable models were evaluated for task admission. Among these, `qwen3:1.7b` achieved the highest reliability and was selected for the admission module. Finally, the complete system was evaluated in a simulated multi-drone environment, demonstrating that the generated plans could be translated into executable drone actions and that unfinished or infeasible subtasks could be returned to the planner for rescheduling.

In summary, this thesis demonstrates that LLMs can serve as a high-level planning interface for heterogeneous multi-drone systems when their outputs are constrained by structured task representations, capability-aware allocation, explicit scheduling, and local feasibility checks.

The proposed architecture shows how natural-language mission planning can be connected to schedule-based replanning, allowing unfinished subtasks to be reassigned when a drone rejects a task or becomes unsuitable during execution.

5.2 Limitations

Although the system demonstrated promising results, several limitations should be considered when interpreting the findings. The most important limitation is the inference time of the LLM-based planning pipeline. Long inference times limit the system's suitability for time-critical missions and missions requiring frequent replanning. While smaller locally deployable models can reduce latency, the results also showed that they are generally less reliable than larger cloud-based models. Similarly, the inference time of the task admission module limits how frequently drone status checks can be performed during execution.

Another limitation is the possibility of invalid LLM outputs. Although structured prompts and validation mechanisms were used, the planner may still occasionally produce malformed JSON outputs, missing fields, or schedules that violate system constraints. Such invalid outputs can disrupt execution because they trigger replanning and thereby delay the mission.

In addition, the generated schedules do not guarantee optimality. As shown in the evaluation, LLM-generated schedules may be valid and executable, but they are not always optimal with respect to the given optimization criterion. This is a limitation compared to classical optimization methods, which can provide stronger guarantees under well-defined assumptions.

5.3 Future Work

Several directions can be explored to improve the proposed system. First, future work should investigate real-world deployment on physical drones. While simulation provides a controlled environment for testing task planning and execution, real-world operation introduces additional challenges such as localization errors, communication delays, wind disturbances, obstacle avoidance, and sensor noise.

Second, the evaluation could be extended with a larger randomized benchmark. In this thesis, the cloud-based planner was evaluated on missions containing one to ten subtasks, with three task descriptions per subtask count. This provides useful initial coverage, but remains limited. A larger benchmark could include more mission descriptions, different object distributions, different drone teams, and varied skill distributions. Such an evaluation would make it possible to assess more reliably whether the observed planning performance generalizes beyond the selected examples.

In addition, the task admission module could be extended with additional constraints, such as weather conditions, no-fly zones, payload limitations, obstacle proximity, and uncertainty in battery consumption. This would make the admission decision more realistic and better suited for deployment in dynamic environments.

Finally, future work could investigate more advanced local deployment of LLMs on embedded drone hardware. This includes optimizing models for hardware acceleration, reducing inference latency, and evaluating whether small onboard models can support not only task admission, but also limited replanning and failure recovery during flight.

In summary, this thesis demonstrates the potential of LLMs as a high-level interface between human operators and autonomous multi-drone systems. With further improvements in validation, real-world testing, and embedded deployment, LLM-based planning could become a useful component in future autonomous drone coordination systems.

Bibliography

[1] R. H. Jacobsen, L. Matlekovic, L. Shi, N. Malle, N. Ayoub, K. Hageman, S. Hansen, F. F. Nyboe, and E. Ebeid, “Design of an autonomous cooperative drone swarm for inspections of safety critical infrastructure,” *Applied Sciences*, vol. 13, no. 3, p. 1256, 2023.

[2] M. A. Dinesh, S. S. Kumar, S. J. Shetty, A. K. N., and M. K. M. Gowda, “Development of an autonomous drone for surveillance application,” *International Research Journal of Engineering and Technology (IRJET)*, vol. 5, no. 8, p. 331, Sep. 2018.

[3] K. Anderson and K. J. Gaston, “Lightweight unmanned aerial vehicles will revolutionize spatial ecology,” *Frontiers in Ecology and the Environment*, vol. 11, no. 3, pp. 138–146, 2013.

[4] A. Ellenberg, A. Kontsos, F. Moon, and I. Bartoli, “Bridge deck delamination identification from unmanned aerial vehicle infrared imagery,” *Automation in Construction*, vol. 72, pp. 155–165, 2016.

[5] M. Erdelj and E. Natalizio, “Uav-assisted disaster management: Applications and open issues,” in *International Conference on Computing, Networking and Communications (ICNC)*. Kauai, HI, USA: IEEE, Feb. 2016, pp. 1–5, hal-01305371.

[6] K. Zhou, Z. Wang, Y.-Q. Ni, Y. Zhang, and J. Tang, “Unmanned aerial vehicle-based computer vision for structural vibration measurement and condition assessment: A concise survey,” *Journal of Infrastructure Intelligence and Resilience*, vol. 2, no. 2, p. 100031, 2023.

[7] Amazon, “Prime air,” https://www.amazon.com/primeair, n.d., accessed: 2026-04-25.

[8] Invest in Denmark, “Denmark launches first long-range drone delivery service,” https://investindk.com/insights/health-drone, May 2022, accessed: 2026-04-25.

[9] P. Nooralishahi, C. Ibarra-Castanedo, S. Deane, F. López, S. Pant, M. Genest, N. P. Avdelidis, and X. P. V. Maldague, “Drone-based non-destructive inspection of industrial sites: A review and case studies,” *Drones*, vol. 5, no. 4, p. 106, 2021.

[10] H. M. Rostum and J. Vásárhelyi, “A review of using visual odometry methods in autonomous uav navigation in gps-denied environment,” *Acta Universitatis Sapientiae, Electrical and Mechanical Engineering*, vol. 15, pp. 14–32, 2023.

[11] L. Wallace, A. Lucieer, Z. Malenovský, D. Turner, and P. Vopěnka, “Assessment of forest structure using two uav techniques: A comparison of airborne laser scanning and structure from motion (sfm) point clouds,” *Forests*, vol. 7, no. 3, p. 62, 2016.

[12] P. McEnroe, S. Wang, and M. Liyanage, “A survey on the convergence of edge computing and ai for uavs: Opportunities and challenges,” *IEEE Internet of Things Journal*, vol. 9, no. 17, pp. 15 435–15 459, 2022.

[13] ModalAI, “Top 5 companion computers for uavs,” 2024. [Online]. Available: https://www.modalai.com/blogs/blog/top-5-companion-computers-for-uavs

[14] M. Wooldridge, *An Introduction to MultiAgent Systems.* Chichester, West Sussex, England: John Wiley & Sons Ltd, 2002, reprinted August 2002.

[15] G. Pantelimon, K. Tepe, R. Carriveau, and S. Ahmed, “Survey of multi-agent communication strategies for information exchange and mission control of drone deployments,” *Journal of Intelligent & Robotic Systems*, vol. 95, no. 3-4, pp. 779–788, 2019.

[16] E. H. Durfee, “Distributed problem solving and planning,” in *Multiagent Systems: A Modern Approach to Distributed Artificial Intelligence*, G. Weiss, Ed. Cambridge, Massachusetts: MIT Press, 1999, pp. 121–164.

[17] T. Bektas, “The multiple traveling salesman problem: An overview of formulations and solution procedures,” *Omega*, vol. 34, no. 3, pp. 209–219, 2006.

[18] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” *arXiv preprint arXiv:1706.03762*, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[19] Y. Kim, D. Kim, J. Choi, J. Park, N. Oh, and D. Park, “A survey on integration of large language models with intelligent robots,” *Intelligent Service Robotics*, vol. 17, pp. 1091–1107, 2024.

[20] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. Jauregui Ruano, K. Jeffrey, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A. Zeng, “Do as i can, not as i say: Grounding language in robotic affordances,” *arXiv preprint arXiv:2204.01691*, 2022. [Online]. Available: https://arxiv.org/abs/2204.01691

[21] W. Wang, Y. Li, L. Jiao, and J. Yuan, “Gsce: A prompt framework with enhanced reasoning for reliable llm-driven drone control,” *arXiv preprint arXiv:2502.12531*, 2025. [Online]. Available: https://arxiv.org/abs/2502.12531

[22] Z. Mandi, S. Jain, and S. Song, "Roco: Dialectic multi-robot collaboration with large language models," 2023. [Online]. Available: https://arxiv.org/abs/2307.04738

[23] S. S. Kannan, V. L. N. Venkatesh, and B.-C. Min, "Smart-llm: Smart multi-agent robot task planning using large language models," *arXiv preprint arXiv:2309.10062*, 2024. [Online]. Available: https://arxiv.org/abs/2309.10062

[24] S. Borate, B. Rai B, V. Pardeshi, and M. Vadali, "Llm-based generalizable hierarchical task planning and execution for heterogeneous robot teams with event-driven replanning," *arXiv preprint arXiv:2511.22354*, 2025. [Online]. Available: https://arxiv.org/abs/2511.22354

[25] T. Yang, P. Feng, Q. Guo, J. Zhang, X. Zhang, J. Ning, X. Wang, and Z. Mao, "Autohma-llm: Efficient task coordination and execution in heterogeneous multi-agent systems using hybrid large language models," *IEEE Transactions on Cognitive Communications and Networking*, vol. 11, no. 2, pp. 987–998, 2025.

[26] OpenAI, "Models," 2026, accessed: 2026-05-01. [Online]. Available: https://developers.openai.com/api/docs/models

[27] Google Developers, "Vehicle routing problem (vrp)," 2026, accessed: 2026-05-01. [Online]. Available: https://developers.google.com/optimization/routing/vrp

[28] Ollama, "Qwen2.5," accessed: 2026-05-01. [Online]. Available: https://ollama.com/library/qwen2.5

[29] ——, "Qwen3," accessed: 2026-05-01. [Online]. Available: https://ollama.com/library/qwen3

[30] ——, "Gemma 3," accessed: 2026-05-01. [Online]. Available: https://ollama.com/library/gemma3

[31] ——, "Llama 3.2," accessed: 2026-05-01. [Online]. Available: https://ollama.com/library/llama3.2

[32] ——, "Phi-4," accessed: 2026-05-01. [Online]. Available: https://ollama.com/library/phi4

[33] Parrot, "Anafi product sheet / white paper," 2021, PDF document, accessed 2026-05-13. [Online]. Available: https://www.parrot.com/assets/s3fs-public/2021-02/anafi-product-sheet-white-paper-en.pdf

[34] ——, "ANAFI Thermal user guide," 2021, user guide, accessed 2026-05-13. [Online]. Available: https://www.parrot.com/assets/s3fs-public/2021-09/anafi-thermal-user-guide.pdf

[35] ——, "ANAFI USA product sheet," 2023, PDF document, accessed 2026-05-13. [Online]. Available: https://www.parrot.com/assets/s3fs-public/2023-02/ANAFI-USA-product-sheet.pdf

[36] ——, "ANAFI Ai product sheet," 2023, PDF document, accessed 2026-05-13. [Online]. Available: https://www.parrot.com/assets/s3fs-public/2023-02/ANAFI-Ai-product-sheet.pdf

[37] A. Sarabakha, "anafi_ros: From off-the-shelf drones to research platforms," *arXiv preprint arXiv:2303.01813*, 2023. [Online]. Available: https://arxiv.org/abs/2303.01813

[38] Vicon Motion Systems, "Tracker Motion Tracking Software," https://www.vicon.com/software/tracker/, accessed: 2026-05-14.